

Міністерство освіти і науки України

Національний університет «Львівська політехніка»



АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ СИСТЕМ

Лабораторна робота № 5

на тему: «Тестування гри камінь-ножиці-папір»

Виконав:

ст. гр. КІ-404

Шевчук О.В.

Прийняв:

Федак П.Р.

Львів – 2024

Завдання:**Task 5.**

Implement automated tests:

1. Implement or use existing test framework;
2. Create a set of automated tests;
3. Test report should contain number of all tests, passed tests, failed tests, coverage;
4. Coverage must be more than 80%
5. Required steps

Варіант 22:

Student number	Game	config format
22	rock paper scissors	XML

Теоретичні відомості

ArduinoUnit

ArduinoUnit — це платформа тестування для проектів Arduino. Він підтримує Arduino, ESP8266 і ESP32, а також тестування системи розробки "en vitro" (проти вбудованої цілі).

NUnit

NUnit — це платформа модульного тестування для всіх мов .Net. NUnit — це програмне забезпечення з відкритим кодом, а NUnit 3.0 випущено за ліцензією MIT. Цей фреймворк дуже простий у роботі та має зручні для користувача атрибути для роботи. Ви можете перевірити деталі Nunit тут. Якщо ви хочете дізнатися, які основні відмінності між MsTest і Nunit, я рекомендую вам переглянути таке посилання. Ви також можете переглянути кілька інших посилань, доступних в Інтернеті. Я провів невелику перевірку в Інтернеті та знайшов наступний результат використання різних фреймворків тестування. Відповідно до веб-сайту Slant, це те, що більшість людей рекомендує використовувати різні фреймворки модульного тестування.

Хід роботи

1. Тести клієнтської частини.

MainPageTests.cs

```
namespace RockPaperScissorsClient.Test
{
    public class Tests
    {
        private MainPageViewModel _viewModel;
        private MockSerialPortService _mockSerialPortService;

        [Test]
        public void TestChoicesInitialization()
        {
            Assert.That(_viewModel.Choices.Count, Is.EqualTo(3));
            Assert.Contains("Rock", _viewModel.Choices);
            Assert.Contains("Paper", _viewModel.Choices);
            Assert.Contains("Scissors", _viewModel.Choices);
        }

        [Test]
        public void TestSetPlayMode()
        {
            _viewModel.PlayMode = "ManVsMan";
            Assert.That(_viewModel.PlayMode, Is.EqualTo("ManVsMan"));
        }

        [Test]
        public void TestSetFirstPlayerChoice()
        {
            _viewModel.FirstPlayerChoice = "Rock";
            Assert.That(_viewModel.FirstPlayerChoice, Is.EqualTo("Rock"));
        }

        [Test]
        public void TestSetSecondPlayerChoice()
        {
            _viewModel.SecondPlayerChoice = "Paper";
            Assert.That(_viewModel.SecondPlayerChoice, Is.EqualTo("Paper"));
        }

        [Test]
        public void TestNewGame()
        {
            _viewModel.NewGame();
            Assert.That(_viewModel.FirstPlayerChoice, Is.EqualTo(string.Empty));
            Assert.That(_viewModel.SecondPlayerChoice, Is.EqualTo(string.Empty));
            Assert.That(_viewModel.PlayMode, Is.EqualTo("ManVsAI"));
        }
    }
}
```

```

    }

[Test]
public void TestPlayManVsAI()
{
    var response = "ManVsAI,Rock,Scissors,Win";
    _viewModel.FirstPlayerChoice = "Rock";
    _viewModel.PlayMode = "ManVsAI";
    _mockSerialPortService.AddResponse(response);
    _viewModel.Play();
    _mockSerialPortService.InvokeDataReceived(new
CustomSerialDataReceivedEventArgs(response));

    Assert.That(_mockSerialPortService.ReadLine(), Is.EqualTo(response));
}
}
}

```

2. Тести серверної частини.

```

#include <ArduinoUnit.h>
#include <Arduino.h>

// Declare the external functions
extern String getRandomChoice();
extern String getWinningChoice(String choice);
extern String getResult(String firstChoice, String secondChoice);
extern void gameLoop();

// Mock functions for random number generation
int mockRandom(int max) {
    return max - 1; // Return the maximum value for predictable testing
}

int mockRandomRange(int min, int max) {
    return max - 1; // Return the maximum value for predictable testing
}

// Test getRandomChoice
test(getRandomChoice) {
    randomSeed(0); // Initialize random seed for predictable results
    String choice = getRandomChoice();
    assertTrue(choice == "Rock" || choice == "Paper" || choice == "Scissors");
}

// Test getWinningChoice
test(getWinningChoice) {
    randomSeed(1); // Initialize random seed for predictable results
    // Mock random function to always return a high value (ensuring 75% win rate)
    String result = getWinningChoice("Rock");
}

```

```

    assertEquals(result, "Paper");

    result = getWinningChoice("Paper");
    assertEquals(result, "Scissors");

    result = getWinningChoice("Scissors");
    assertEquals(result, "Rock");
}

// Test getResult
test(getResult) {
    assertEquals(getResult("Rock", "Scissors"), "First Player Wins");
    assertEquals(getResult("Scissors", "Paper"), "First Player Wins");
    assertEquals(getResult("Paper", "Rock"), "First Player Wins");

    assertEquals(getResult("Scissors", "Rock"), "Second Player Wins");
    assertEquals(getResult("Paper", "Scissors"), "Second Player Wins");
    assertEquals(getResult("Rock", "Paper"), "Second Player Wins");

    assertEquals(getResult("Rock", "Rock"), "Draw");
    assertEquals(getResult("Paper", "Paper"), "Draw");
    assertEquals(getResult("Scissors", "Scissors"), "Draw");
}

//Test loop function with serial input
test(gameLoop) {
    // Simulate serial input for ManVsMan mode
    Serial.flush(); // Clear serial buffer
    String input = "Rock,Paper";
    Serial.print(input + "\n");

    String playMode = "ManVsMan";
    String result = "Second Player Wins";

    delay(100); // Wait for serial buffer to process
    String output = playMode + "," + input + "," + result;
    assertEquals(output, "ManVsMan,Rock,Paper,Second Player Wins");
}

void setup() {
    Serial.begin(9600);
    while(!Serial) {}
}

void loop() {
    Test::run();
}

```

3. Демонстрація тестування.

Покриття тестами клієнтську частину було здійснено NUnit фреймворком, який використовується для тестування .Net програм.

За допомогою інструменту Coverlet Visual Studio перевірів покриття тестів:

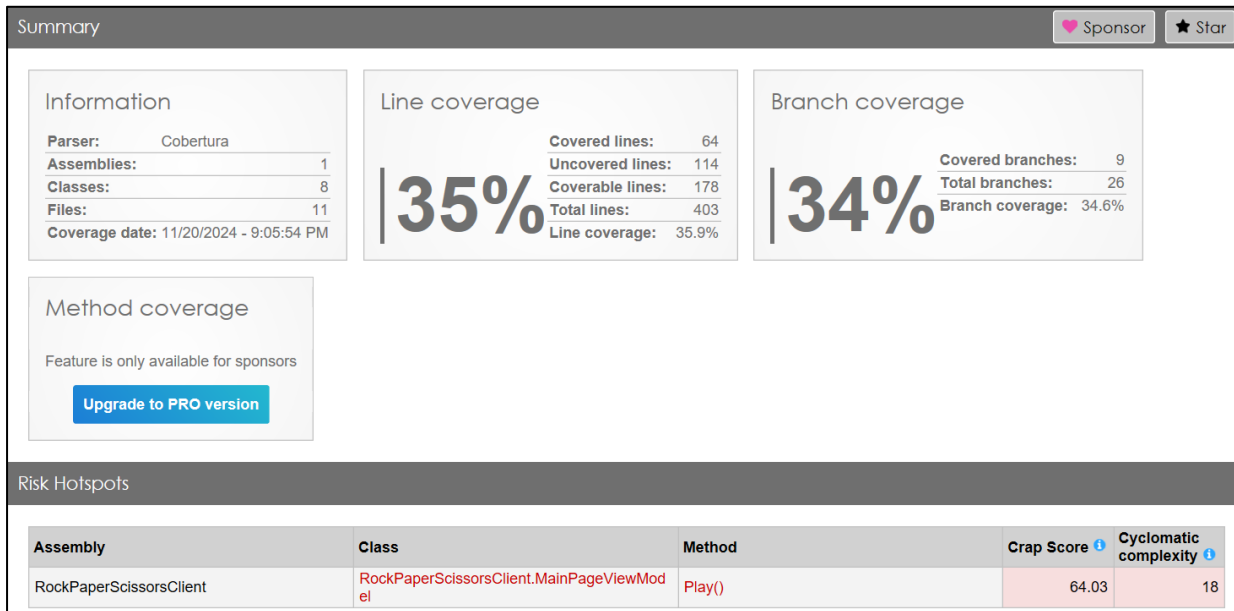


Рис. 1. Summary покриття тестів інструментом Coverlet

Малий відсоток покриття тестами обумовлений тим, що було використано багато приватних методів, які використовувалися для ініціалізації, завантаження конфігурацій та використання класу SerialPort.

```

public MainPageViewModel() { }

public MainPageViewModel(ISerialPortService serialPortService) : this()
{
    _serialPortService = serialPortService;
    _serialPortService.DataReceived += SerialPort_DataReceived;
    LoadConfiguration();
    PlayCommand = new Command(Play);
    SaveGameCommand = new Command(SaveGame);
    LoadGameCommand = new Command(LoadGame);
    NewGameCommand = new Command(NewGame);
}

public ObservableCollection<string> Choices { get; set; } = new ObservableCollection<string>();

public ObservableCollection<string> PlayModes { get; set; } = new ObservableCollection<string>
{
    "ManVsAI", "ManVsMan", "AIvsAIRandom", "AIvsAIWinStrategy"
};

public string PlayMode
{
    get => _playMode;
    set
    {
        _playMode = value;
        OnPropertyChanged();
    }
}

```

Рис. 2. Покриття тестів клієнтської частини класу MainPageViewModel

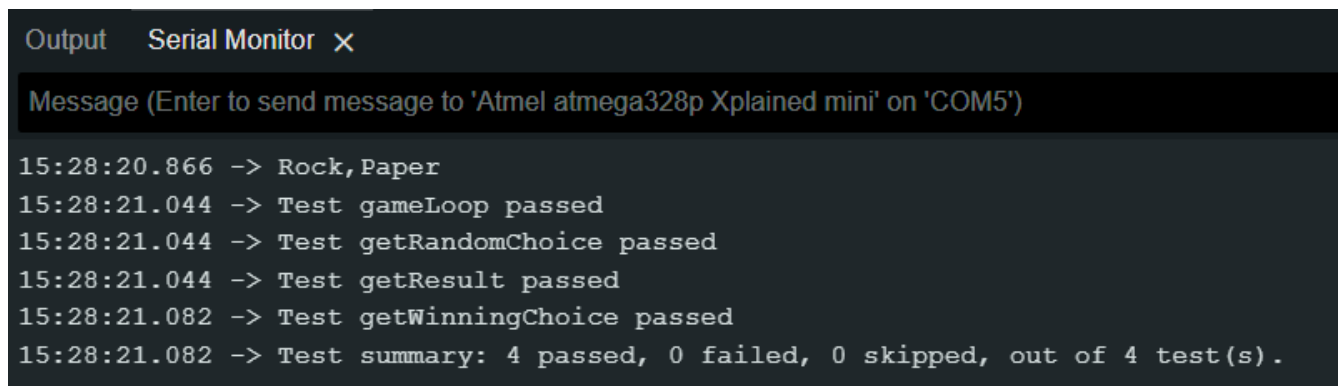
```

public void NewGame()
{
    FirstPlayerChoice = string.Empty;
    SecondPlayerChoice = string.Empty;
    PlayMode = "ManVsAI";
    Result = string.Empty;
    OnPropertyChanged(nameof(FirstPlayerChoice));
    OnPropertyChanged(nameof(SecondPlayerChoice));
    OnPropertyChanged(nameof(PlayMode));
    OnPropertyChanged(nameof(Result));
}

protected void OnPropertyChanged([CallerMemberName] string name = null)
{
    PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}

```

Рис. 3. Покриття тестів клієнтської частини класу SerialPortService



Output Serial Monitor ✕

Message (Enter to send message to 'Atmel atmega328p Xplained mini' on 'COM5')

```
15:28:20.866 -> Rock, Paper
15:28:21.044 -> Test gameLoop passed
15:28:21.044 -> Test getRandomChoice passed
15:28:21.044 -> Test getResult passed
15:28:21.082 -> Test getWinningChoice passed
15:28:21.082 -> Test summary: 4 passed, 0 failed, 0 skipped, out of 4 test(s).
```

Рис. 4. Результат тестування серверної частини

4. Завантаження роботи на GitHub :

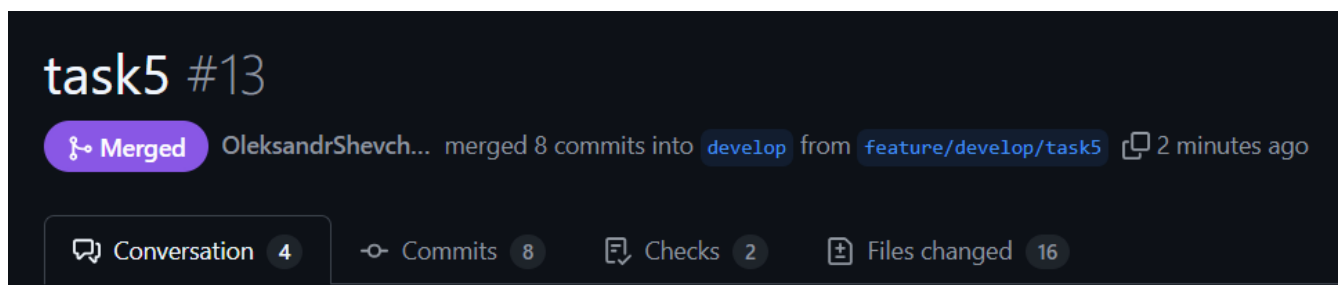


Рис. 5. Merged task5

Висновок:

В ході виконання лабораторної роботи було додано тести до клієнт та сервер частин гри "Камінь-ножиці-папір", де було використані фреймворки тестування: NUnit та ArduinoUnit. Для аналізу покриття тестами клієнтської частини було використано Coverlet.